

A comparative study of Bitcoin and Ripple cryptocurrencies trading using Deep Reinforcement Learning algorithms

Dieu-Donné Fangnon^{a,b,1}, Armandine Sorel Kouyim Meli^{a,b,1}, Verlon Roel Mbingui^{a,b,1}, Phanie Dianelle Negho^{a,b,1} and Regis Konan Marcel Djaha^{a,b,1}

^aAfrican Masters of Machine Intelligence (AMMI)

^bAfrican Institute for Mathematical Sciences (AIMS), Senegal

Abstract. Artificial intelligence (AI) has demonstrated remarkable success across various applications. In light of this trend, the field of automated trading has developed a keen interest in leveraging AI techniques to forecast the future prices of financial assets. This interest stems from the need to address trading challenges posed by the inherent volatility and dynamic nature of asset prices. However, crafting a flawless strategy becomes a formidable task when dealing with assets characterized by intricate and ever-changing price dynamics. To surmount these formidable challenges, this research employs an innovative rule-based strategy approach to train Deep Reinforcement Learning (DRL). This application is carried out specifically in the context of trading Bitcoin (BTC) and Ripple (XRP). Our proposed approach hinges on the integration of Deep Q-Network, Double Deep Q-Network, Dueling Deep Q-learning networks, alongside the Advantage Actor-Critic algorithms. Each of them aims to yield an optimal policy for our application. To evaluate the effectiveness of our Deep Reinforcement Learning (DRL) approach, we rely on portfolio wealth and the trade signal as performance metrics. The experimental outcomes highlight that Dueling and Double Deep Q-Network outperformed when using XRP with the increasing of the portfolio wealth. All codes are available in this [Github link](#).

Keywords: Deep Reinforcement Learning, Cryptocurrency, Trading

1 Introduction

Cryptocurrency markets are notoriously volatile and complex, making them a difficult but appealing playground for algorithmic trading. In recent years, Deep Reinforcement Learning (DRL) has emerged as a promising approach to mastering the intricacies of cryptocurrency trading. DRL methods harness the combined strengths of neural networks and reinforcement learning to enable agents to learn effective trading strategies directly from market data [11]. This comparative analysis explores the dynamic intersection of cryptocurrency trading and innovative DRL approaches. It provides a holistic review of diverse DRL methods and their relevance in cryptocurrency markets. As digital assets ascend in the global financial landscape, traders and investors are increasingly embracing AI-powered DRL trading

strategies. The central goal of this study is to assess the performance, robustness, and adaptability of different DRL algorithms in cryptocurrency trading settings. To this end, we scrutinize a spectrum of DRL approaches, including Deep Q-Networks (DQN) [15], Double Deep Q-Networks [19], Dueling Deep Q-Networks [20], and Advantage Actor-Critic (A2C) [6]. Each method's strengths, weaknesses, and suitability for cryptocurrency trading will be rigorously evaluated. Furthermore, this study addresses practical considerations such as data preprocessing, feature engineering, risk management, and model hyperparameters to deliver a comprehensive assessment. By contrasting and comparing these DRL approaches, we aspire to offer insightful insights into their potential to augment trading strategies in the volatile and lucrative cryptocurrency realm. The insights from this comparative analysis can be a valuable asset for traders, investors, and researchers seeking to leverage the power of DRL for cryptocurrency trading. Additionally, it advances the wider dialogue on the convergence of AI and machine learning in financial markets, shedding light on the evolving algorithmic trading landscape in the digital age.

Our work is structured as follows: In Section 2, we provide a literature review. Section 3 covers the methodology, where we present the formalization of the Markov Decision Process, RL algorithms. In Section 4, we discuss the experiments, starting with a description of the dataset, followed by the presentation of the environment, and present the results.

2 Literature Review

In this section, we review some classical trading strategies and discuss how RL has been applied to this field.

Algorithmic trading is a systematic methodology characterized by mathematical modeling and automated execution. It encompasses a variety of trading strategies, such as trend-following [13], mean-reversion [4], statistical arbitrage [3], and delta-neutral trading strategies [8]. In this context, our primary focus is on the evaluation of time series momentum strategies presented in the work of [23], which serves as a benchmark for our models.

The research conducted by [23] has produced an exceptionally robust trading strategy, simply based on utilizing the sign of returns over the preceding year as a signal. Their study demonstrated the profitability of this approach across a span of 25 years, encompass-

¹ Equal contribution.

ing 58 different liquid financial instruments.

The existing literature on Reinforcement Learning (RL) in trading can be broadly classified into three primary methodologies: critic-only, actor-only, and actor-critic approaches [17]. Notably, the critic approach, predominantly implemented using Deep Q-Networks (DQN), has garnered the most attention in this domain [7, 5]. This approach involves the construction of a state-action value function, denoted as Q , which quantifies the quality of a specific action within a given state.

Actor-only approaches directly optimize the objective function without the need to compute the expected outcomes of each action in a given state. This direct policy learning makes actor-only methods versatile and applicable to continuous action spaces. Notably, in the research conducted by [16, 12], offline batch gradient ascent techniques are employed to optimize objective functions like profits or the Sharpe ratio. These approaches are advantageous because they offer an end-to-end differentiable optimization process.

It's important to distinguish this from standard RL actor-only approaches, where the focus is on learning a policy distribution. In these cases, the Policy Gradient Theorem [1] and Monte Carlo methods [14] come into play during training. Models are updated iteratively, typically at the end of each episode, in order to study and refine the distribution of the policy. This distinction highlights the various strategies employed in actor-only RL methods, depending on the specific objectives and challenges encountered in trading scenarios.

The actor-critic approach represents the third category of RL methodologies and addresses the challenges posed by real-time policy updates. This approach hinges on a fundamental concept: the simultaneous updating of two distinct models. The "actor" model governs an agent's actions based on the current state, while the "critic" model assesses the quality or goodness of the chosen actions.

However, it's worth noting that within financial applications, the actor-critic approach has received relatively limited attention compared to other methods. There have been fewer studies in this domain, with only a few notable works, such as those by [1, 11], exploring its potential and applicability. Despite being less studied, the actor-critic approach holds promise for addressing real-time learning challenges in financial contexts.

3 Methodology

We present several configurations, which include state and action spaces as well as reward functions. In our study, we will employ four reinforcement learning (RL) algorithms: Deep Q Networks, Double Deep Q Networks, Dueling Deep Q Networks and Deterministic Deep Q Networks.

3.1 Markov Decision Process Formalisation

We can frame the trading problem as a Markov Decision Process (MDP) in which an agent engages with the environment during discrete time intervals. At each time step t , the agent is provided with a representation of the environment referred to as a state S_t . Given this state, the agent selects an action A_t , and as a consequence of this action, a numerical reward R_{t+1} is assigned to the agent at the subsequent time step, placing the agent in a new state S_{t+1} . The interaction between the agent and the environment generates a trajectory $\tau = [S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, \dots]$. At any given time step t , the objective of Reinforcement Learning (RL) is to maximize the expected return, denoted as G_t at time t [22], which essentially represents the expected cumulative rewards, often discounted :

$$G_t = \sum_{k=t+1}^T \gamma^{k-t-1} R_k \quad (1)$$

When the discounting factor, denoted as γ , is considered, optimizing the expectation $\mathbb{E}(G)$ is equivalent to optimizing our expected wealth if the utility function in Equation 1 takes a linear form and we use R_t to represent trade returns.

3.2 RL Algorithms

In this section, we present some algorithms used in this work.

Deep Q Networks

Deep Q-learning Networks (DQN), employ a neural network to approximate the state-action value function, also known as the Q function. This Q function is used to estimate how advantageous it is for the agent to take a specific action in a particular state [15]. Assuming that our Q function is represented by a set of parameters denoted as θ , our objective is to minimize the mean squared error between the current Q value and the target Q value. This minimization process leads to the derivation of the optimal state-action value function.

$$L(\theta) = \mathbb{E} [(Q_\theta(S, A) - Q_{\theta'}(S, A))^2] \quad (2)$$

$$Q_{\theta'}(S_t, A_t) = r + \gamma \max_{A'} Q_{\theta'}(S_{t+1}, A_{t+1})$$

where $L(\theta)$ is the objective function. A problem is that the training of a vanilla DQN is not stable and suffers from variability. Many improvements have been made to stabilise the training process.

Double Deep-Q Networks

The DQN algorithm employs a max operator to estimate the Q -target, deliberately opting for the highest value. This approach not only executes the action but also assesses it based on this particular methodology. However, it has been demonstrated that the selection and evaluation of the highest value tend to be overly optimistic, potentially leading to training stagnation. To address this issue, the Double Deep Q-Network (DDQN) [18] introduces a separation between action selection and evaluation. In this revised process, the action is taken based on a network with parameters represented as θ , while the action is evaluated using a separate network with parameters denoted as θ' that considers the next state. This distinction can be formally expressed as follows [9]:

$$y_i = r + \gamma Q \left(s', \underset{a' \in A}{\operatorname{argmax}} Q(s', a, \theta_i); \theta'_i \mid s, a \right). \quad (3)$$

The DQN algorithm already uses a second network (target network) with weights θ' , which can be viewed as a natural choice for the DDQN algorithm. In conclusion, the DDQN is an extension of the DQN, with the key feature that it additionally uses the target network to separate the execution and evaluation process of an action.

Dueling Deep Q Networks

The Agent's underlying algorithm is the core module of the RL setup. To simplify, the RL agent learns the sequence of actions that maximize an objective function instead of minimizing it as in the case of

a typical deep learning pipeline [20]. We do it recursively using the Bellman Equation.

$$Q(s, a; \theta) = r + \gamma Q(s', \arg\max_{a'} Q(s', a'; \theta); \theta')$$

Bellman Equation for Deep Q Networks The $Q(s, a; \theta)$ denotes the maximum expected future reward for choosing action a in state s . The Q -value is constantly updated through an iterative process. Deep Q Networks comprise neural networks acting as function approximations for Q-Table. Inside a DQN, the neural network takes the state as input and outputs the Q -value for each action. The action with the maximum value is then chosen and communicated back to the environment. Dueling DQN is an extension of Deep Q Networks which includes the calculation of the Advantage of action over other actions in the final output layer[20].

$$Q(s, a) = V(s) + \left(A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a') \right) \quad (4)$$

Advantage Actor-Critic (A2C)

The A2C is proposed to solve the training problem of PG by updating the policy in real-time. It consists of two models: one is an actor network that outputs the policy and the other is a critic network that measures how good the chosen action is in the given state [6]. We can update the policy network $\pi(A | S, \theta)$ by maximising the objective function:

$$J(\theta) = \mathbb{E} [\log \pi(A | S, \theta) A_{adv}(S, A)] \quad (5)$$

where $A_{adv}(S, A)$ is the advantage function defined as:

$$A_{adv}(S_t, A_t) = R_t + \gamma V(S_{t+1} | w) - V(S_t | w) \quad (6)$$

In order to calculate advantages, we use another network, the critic network, with parameters w to model the state value function $V(s | w)$, and we can update the critic network using gradient descent to minimize the TD-error:

$$J(w) = (R_t + \gamma V(S_{t+1} | w) - V(S_t | w))^2 \quad (7)$$

The A2C is most useful if we are interested in continuous action spaces as we recude the policy variance by using the advantage function and update the policy in real-time. The training of A2C can be done synchronously or asynchronously (A3C).

4 Experiments

4.1 Description of Dataset

The data used in this work was downloaded from the yahoo finance. The dataset is a financial dataset containing daily stock market data for multiple assets such as equities, ETFs, and indexes. It spans from August 30, 2015 to August 30, 2023, and contains 1257 rows and 7 columns namely:

- Date: The date on which the stock market data was recorded.
- Open: The opening price of the asset on the given date.
- High: The highest price of the asset on the given date.
- Low: The lowest price of the asset on the given date.
- Close: The closing price of the asset on the given date.
- Adj Close: The adjusted closing price of the asset on the given date.
- Volume: The total number of shares of the asset that were traded on the given date.

To train the different models, we have divided our datasets into Two parts: 0.9 for the training and 10 for the test.

4.2 Environment

We will now proceed to define the three primary attributes of the trading agent: the state space, the action space, and the reward function.

State Space

In the realm of literature, various attributes have been employed to define state spaces. Notably, historical price data of a security is a consistent inclusion, alongside frequent utilization of associated technical indicators [18]. In our research, we adopt a state representation that encompasses historical prices, returns (r_t) computed across different time horizons, and technical indicators, including Moving Average Convergence Divergence (MACD) [2] and the Relative Strength Index (RSI) [21]. For each specific time step, we aggregate the past 60 observations for each of these features to create a unified state. Here is a list of the features we incorporate:

- Normalised close price series,
- Returns over the past month, 2-month, 3-month and 1-year periods are used. Following [10], we normalise them by daily volatility adjusted to a reasonable time scale. As an example, we normalise annual returns as $r_{t-252:t} / (\sigma_t \sqrt{252})$ where σ_t is computed using an exponentially weighted moving standard deviation of r_t with a 60-day span,
- MACD indicators are proposed in [2] where:

$$\text{MACD}_t = \frac{q_t}{\text{std}(q_{t-252:t})}$$

$$q_t = (m(S) - m(L)) / \text{std}(p_{t-63:t})$$

where $\text{std}(p_{t-63:t})$ is the 63-day rolling standard deviation of prices p_t and $m(S)$ is the exponentially weighted moving average of prices with a time scale S ,

- The RSI is an oscillating indicator moving between 0 and 100. It indicates the oversold (a reading below 20) or overbought (above 80) conditions of an asset by measuring the magnitude of recent price changes. We include this indicator with a look back window of 30 days in our state representations.

Action Space

The action space defines the spectrum of actions available to our agent based on the state representation. These actions are as follows:

- -1 = Sell the asset,
- 0 = Take no action,
- 1 = Buy the asset .

The agent conveys its intention to either Buy or Sell by selecting a value from the set $\{-1, 0, 1\}$ as defined above. The method used to interpret these action values depends on the specific algorithm employed by the agent, which we will delve into in greater detail in the subsequent section dedicated to the Agent.

Reward Function

Running Rewards: Running Rewards are given by the environment to the agent as long as the state is non-terminal. The environment rewards the agent based on the action it takes. Let's demonstrate this with an example:

- Let the future return for the time period t be $r(t)$
- $A(t)$ be the agent's action at time t . $A(t)$ can take the values $\{-1, 0, 1\}$
- $S(t)$ be the vector representation of the state for time t

Then the reward $R(t)$ which will be received by the agent after taking action $A(t)$ on observing $S(t)$ can be computed as:

$$R(t) = r(t) * A(t) - |(A(t) - A(t-1))| * C \quad (8)$$

Where $r(t)$ is the future return for the asset and C represents transaction costs which we are assuming will be in the range of 1 – 5 basis points per trade. (1 basis point = 1% of 1% = 0.0001). For example, if $r(t)$ is positive(negative) and the agent chooses $A(t)$ as 1(-1), i.e., $r(t)$ and $A(t)$ have the same sign, then the return is positive and the agent is rewarded. Conversely, if $r(t)$ and $A(t)$ have opposite signs, i.e., if the future return is negative and the agent decides to buy, then the reward will be negative and the agent will be punished with a negative reward. This is a gross oversimplification but it is fundamentally how reinforcement learning agent learns. As the name suggests, the environment Reinforces the decisions made by the agent through positive and negative rewards.

Terminal Rewards: These are the rewards given by the environment when the agent completes the task that is it reaches the terminal state. The rewards given depends on how the Agent reached the terminal state.

- If the agent uses up **70%** of the capital, then that is not a very favourable situation for us. So we heavily punish the agent by giving a large negative reward.
- If the agent reaches the end of the episode with enough capital, then we will present the agent with a multiple of the final portfolio return. If the final portfolio return is positive, the reward is highly positive and we are teaching the agent that it is learning in the profitable direction. Same for negative returns except that here we are punishing the agent with a large negative reward which will tell the agent to change its strategy.
- We can also use a higher negative multiplier to make the agent more riskaverse towards a negative return.

4.3 Experimental Results

Let's now assess our agent's effectiveness by providing him with an initial capital of about \$100,000. Plots of the signal generated and the portfolio wealth accumulated in the test conditions are shown below. The subplot on the left side displays the trading signals (buy/sell) created by the agent for each of the coins used, including XRP-USD and Bitcoin (BTC-USD), while the subplot on the right side represents the value of the portfolio for different models.

4.3.1 Deep Q Network (DQN)

Here we have trained the agent using the DQN model [1,2]. The results of the above graphs show that, the agent generate more profits with XRP asset than with the Bitcoin cryptocurrency. This conclusion is based on our observation of numerous successful sales and purchases along the signal curve. These transactions have led to substantial gains, as reflected in our portfolio's growth beyond the initial capital investment.

However, our assessment also reveals fluctuations in the portfolio's value, notably in the vicinity of the initial capital. These variations serve as a testament to the agent's ability to generate both profits and losses during specific time periods.

From November 2022 to January 2023, we observed significant fluctuations in the portfolio, demonstrating the agent's capacity to navigate through volatile market conditions. Similarly, between May 2023 and the present, we observed a notable decline in the value of Bitcoin (BTC), underscoring the challenges and opportunities presented by the cryptocurrency market in 2023. These results shows that our model performs better with XRP-USD asset than with BTC-USD.

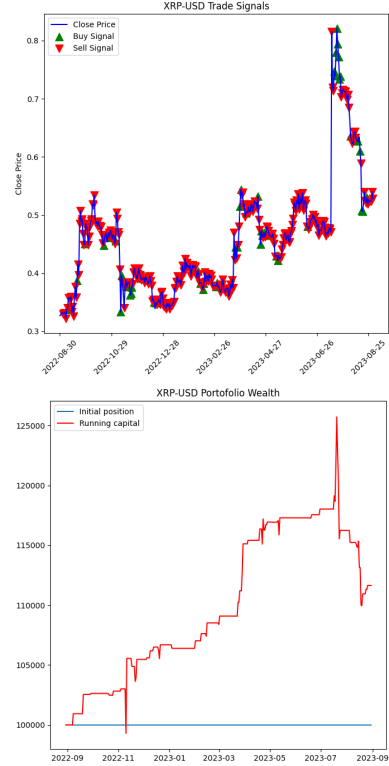


Figure 1. XRP USD

4.3.2 Double Deep Q Network (DDQN)

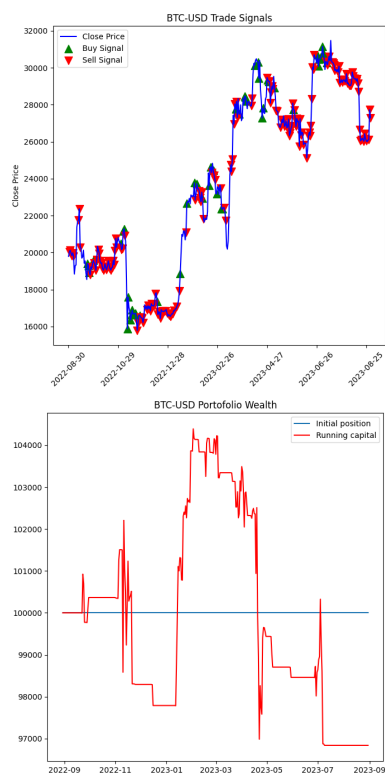


Figure 2. Bitcoin USD

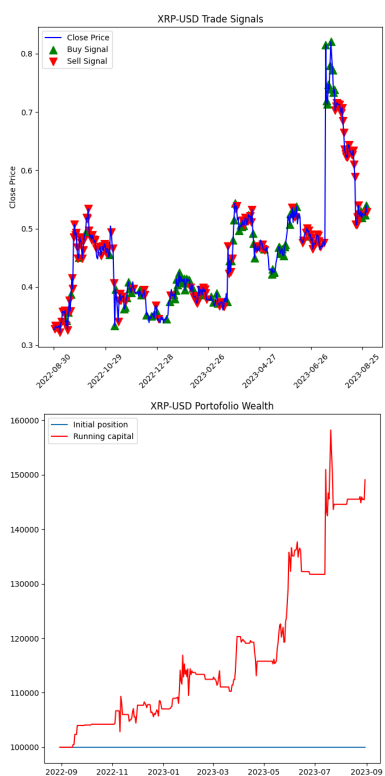


Figure 3. XRP USD

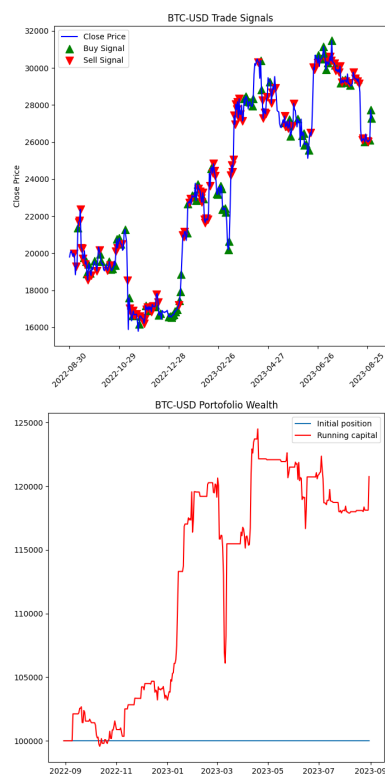


Figure 4. Bitcoin USD

The provided graphs above [3,4] illustrate the performance of the agent under Double DQN model using Ripple (XRP) and BTC as assets. Firstly, the Ripple portfolio exhibits remarkable growth, 25% of the initial capital over time, indicating a highly successful investment strategy. Moreover, Ripple's graph indicates consistent gains without any visible losses until 2023. In contrast to the first model, here we see that the agent does generate profits on the initial capital by using Bitcoin with a positive reward. We can therefore conclude that for BTC trading, it is preferable to train an agent with the Double DQN model rather than the single DQN.

4.3.3 Dueling Deep Q Network

The figures [5,6] below show the performance of the agent trained with a new model, the Dueling DQN. With this model, we can clearly see that the agent generates a lot of increasing benefits using XRP. Ripple demonstrates impressive and exponential growth over time, highlighting its potential as a lucrative investment option. Although there were initial losses for several months, the agent's perseverance eventually paid off, showcasing XRP's ability to recover and generate substantial gains. XRP's growth trajectory suggests that it could be one of the best-performing cryptocurrencies, providing investors with opportunities for significant returns. But with Bitcoin, the agent fails to generate profits over the entire period under consideration. At the beginning, the agent generated a large profit, but by May 2023, the initial capital had fallen completely. This downturn serves as a reminder of the volatile nature of cryptocurrencies, where rapid fluctuations can impact investment outcomes. Also, may be due to the volatility of BTC in 2023, the fall in its dollar value on the asset market. So to trade BTC using a Dueling agent, it would be preferable to use data before 2023. But there's no problem with XRP.

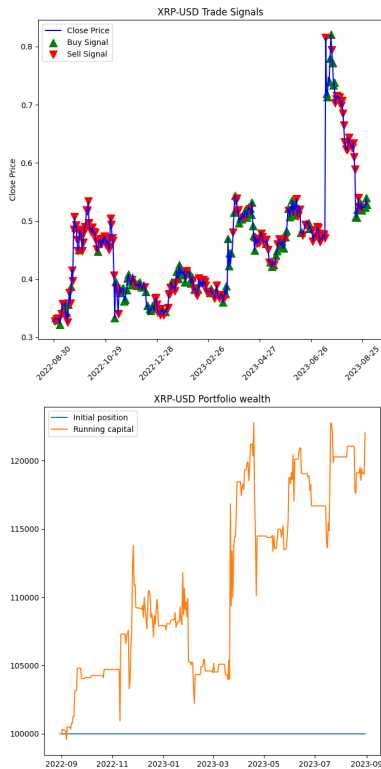


Figure 5. Ripple USD

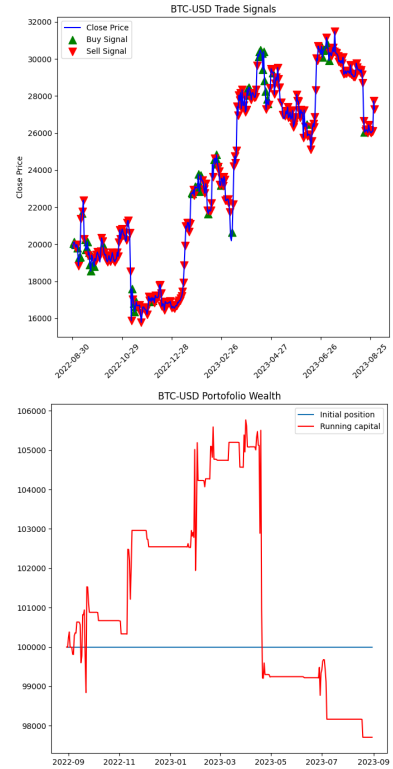


Figure 6. Bitcoin USD

4.3.4 Advantage Actor-Critic (A2C)

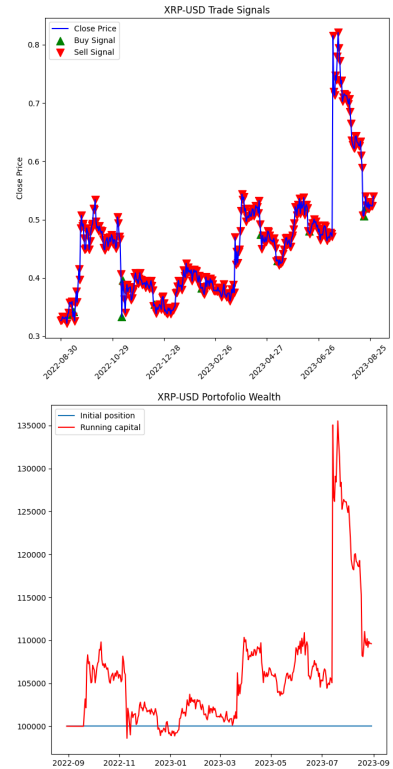


Figure 7. XRP-USD

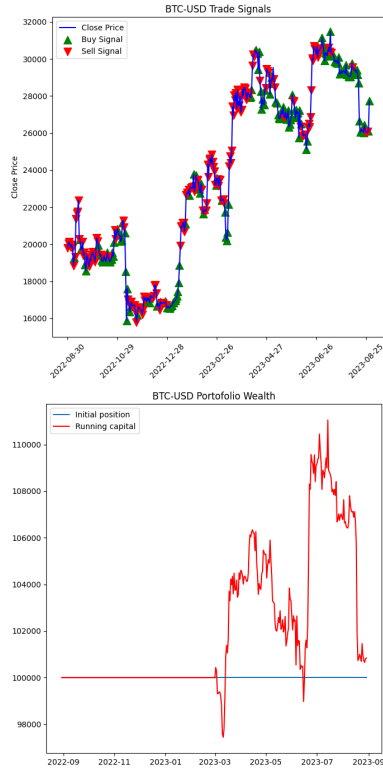


Figure 8. BTC-USD

In this case [7,8], the agent was trained using the A2C model. It can be seen from the XRP graphs above (8) that the agent sold more than it bought, suggesting that the agent did not generate much profit at the outset. In July 2023, the portfolio grew exponentially before falling sharply again. Compared with other models, A2C is not well suited to XRP trading. In the case of Bitcoin (8), the agent took no action at the beginning, but capital decreased in March 2023 before growing again and falling again. Of the two models, we can see that the model is better suited to XRP trading than to Bitcoin.

5 Conclusion

In this paper, we delved into the world of cryptocurrency trading, aiming to enhance trading strategies by harnessing the power of deep reinforcement learning (RL). We employed four RL models, namely DQN, Double DQN, Dueling DQN, and A2C, to scrutinize their efficacy in optimizing trading decisions. Our investigation encompassed two prominent cryptocurrencies, XRP and Bitcoin, allowing us to gain insights into the models' performance across different assets. Throughout our experiments, we uncovered valuable insights into the capabilities of these RL agents. Notably, our findings indicated that these models demonstrated superior performance when applied to XRP trading in comparison to Bitcoin. The assessment of their effectiveness was facilitated by the visualization of portfolio wealth plots, where each agent was entrusted with an initial capital and tasked with accumulating profits. Our results underscored the adaptability and potential of deep RL methods in the realm of cryptocurrency trading. While Bitcoin, often regarded as a flagship digital asset, presented challenges and complexities that proved to be formidable for our agents, XRP exhibited a more favorable environment for these models to thrive. It is imperative to acknowledge that the cryptocurrency market is highly dynamic and influenced by multifarious fac-

tors, making it a challenging domain for trading algorithms. The differential performance of the RL models across assets highlights the importance of tailoring strategies to the unique characteristics of individual cryptocurrencies.

- The originality of our work in relation to other work on the same subject lies in the fact that we have not only tested the performance of an agent on the financial market for trading by implementing four different models, but we have also extended the range of our data to 2023 (this is a new research study). We were able to show that models such as Double DQN and Dueling DQN perform well for XRP, while for Bitcoin it's Double DQN. This work then forms the basis for future research into the financial market for cryptocurrency trading.

To even improve the performance of the agent, we can do some changes as follows:

- Use an LSTM Encoder architecture with Attention to extract the features from the time-series data and then pass the feature vector to the agent as input instead of the existing architecture ; Choosing a policy-based method such as the Deep Deterministic Policy Gradient algorithm to specify the amount of asset to buy or sell instead of just going maximally long or short with the investment.

References

- [1] A. Balakrishnan, S. Jaksic, E. Aguilar, D. Nickovic, and J. Deshmukh. Model-free reinforcement learning for symbolic automata-encoded objectives. In *Proceedings of the 25th ACM International Conference on Hybrid Systems: Computation and Control*, pages 1–2, 2022.
- [2] J. Baz, N. Granger, C. R. Harvey, N. Le Roux, and S. Rattray. Dissecting investment strategies in the cross section and time series. *Available at SSRN 2695101*, 2015.
- [3] Y. Değirmencioglu and İ. Z. Akyurt. Forecasting. *Smart and Sustainable Operations and Supply Chain Management in Industry 4.0*, 2023.
- [4] K. Fatah and T. Nazar. Hierarchical portfolio allocation with community detection, 2022.
- [5] F. Giorgi, S. Herzel, and P. Pigato. Reinforcement learning for investment strategies with trading signals and transaction costs.
- [6] S. Huang, A. Kanervisto, A. Raffin, W. Wang, S. Ontañón, and R. F. J. Dossa. A2c is a special case of ppo. *arXiv preprint arXiv:2205.09123*, 2022.
- [7] Y. Huang, Y. Jia, and X. Zhou. Achieving mean–variance efficiency by continuous-time reinforcement learning. In *Proceedings of the Third ACM International Conference on AI in Finance*, pages 377–385, 2022.
- [8] M.-C. Hung, P.-H. Hsia, X.-J. Kuang, and S.-K. Lin. Intelligent portfolio construction via news sentiment analysis. *International Review of Economics & Finance*, 89:605–617, 2024.
- [9] J. D. Lafferty, C. K. I. Williams, J. Shawe-Taylor, R. S. Zemel, and A. Culotta, editors. *Advances in Neural Information Processing Systems 23: 24th Annual Conference on Neural Information Processing Systems 2010. Proceedings of a meeting held 6-9 December 2010, Vancouver, British Columbia, Canada*, 2010. Curran Associates, Inc.
- [10] B. Lim, S. Zohren, and S. Roberts. Enhancing time-series momentum strategies using deep neural networks. *The Journal of Financial Data Science*, 2019.
- [11] X.-Y. Liu, Z. Xiong, S. Zhong, H. Yang, and A. Walid. Practical deep reinforcement learning approach for stock trading. *arXiv preprint arXiv:1811.07522*, 2018.
- [12] C. I. Lu. Evaluation of deep reinforcement learning algorithms for portfolio optimisation. *arXiv preprint arXiv:2307.07694*, 2023.
- [13] V. P. Mandal's. Kg joshi college of arts & ng bedekar college of commerce, thane (autonomous). 2022.
- [14] N. Metropolis and S. Ulam. The monte carlo method. *Journal of the American statistical association*, 44(247):335–341, 1949.
- [15] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.
- [16] D. Poh, B. Lim, S. Zohren, and S. Roberts. Enhancing cross-sectional currency strategies by context-aware learning to rank with self-attention. *The Journal of Financial Data Science*, 2022.

- [17] S. Sun, R. Wang, and B. An. Reinforcement learning for quantitative trading. *ACM Transactions on Intelligent Systems and Technology*, 14 (3):1–29, 2023.
- [18] H. van Hasselt, A. Guez, and D. Silver. Deep reinforcement learning with double q-learning, 2015.
- [19] H. Van Hasselt, A. Guez, and D. Silver. Deep reinforcement learning with double q-learning. In *Proceedings of the AAAI conference on artificial intelligence*, volume 30, 2016.
- [20] Z. Wang, T. Schaul, M. Hessel, H. Hasselt, M. Lanctot, and N. Freitas. Dueling network architectures for deep reinforcement learning. In *International conference on machine learning*, pages 1995–2003. PMLR, 2016.
- [21] J. W. Wilder. New concepts in technical trading systems. (*No Title*), 1978.
- [22] Z. Zhang, S. Zohren, and R. Stephen. Deep reinforcement learning for trading. *The Journal of Financial Data Science*, 2020.
- [23] S. Zohren, S. Roberts, X. Dong, et al. Learning to learn financial networks for optimising momentum strategies. *arXiv preprint arXiv:2308.12212*, 2023.